

07 – Project Documentation

Client Script & UI Policy in ServiceNow Incident Management

Project Title	Client Script & UI Policy in ServiceNow Incident Management
Technology Track	ServiceNow Administrator
Platform	ServiceNow
Application Module	Incident Management
Main Components	Client Script and UI Policy

1. Introduction

The project titled **Client Script & UI Policy in ServiceNow Incident Management** is a ServiceNow Administrator project focused on managing the behaviour of fields in the Incident form. The project uses two important ServiceNow configuration features, Client Script and UI Policy, to provide dynamic behaviour according to configured conditions.

The Incident form is an important interface through which users create and manage incident records. A well-configured form can help users enter relevant information in an organized manner and can reduce unnecessary manual actions. In this project, the form behaviour is designed around conditions so that relevant fields can respond according to the configuration.

2. Project Background

In a basic form, users may have to manually understand which fields are relevant for a particular situation and may need to manage field values or properties themselves. This can make the process less convenient and can lead to unnecessary interaction with fields that are not applicable to the current condition.

ServiceNow provides configuration capabilities that allow form behaviour to be controlled on the client side. Client Scripts can provide client-side logic, while UI Policies can dynamically control field properties. The combination of these features forms the basis of this project.

3. Problem Statement

The project addresses the requirement of managing Incident form fields according to specific conditions. Without appropriate dynamic configuration, users may have to perform additional manual actions or may not receive clear field behaviour when Incident information changes.

Therefore, the project proposes a ServiceNow-based approach in which Client Script and UI Policy are configured to control the relevant Incident form behaviour. The resulting form should provide a more structured and user-friendly interaction.

4. Project Purpose

The purpose of this project is to control and manage Incident form fields based on specific conditions. Client Script is used to provide client-side behaviour, while UI Policy is used to dynamically control field properties in the Incident form.

- Provide dynamic behaviour within the Incident form.
- Control relevant field properties according to configured conditions.
- Reduce unnecessary manual actions for users.

- Make the Incident form more structured and user-friendly.
- Demonstrate practical ServiceNow Administrator configuration.

5. Project Objectives

No.	Objective	Description
1	Implement Client Script	Configure client-side behaviour for the Incident form according to the project requirement.
2	Implement UI Policy	Configure dynamic field properties based on the applicable condition.
3	Manage field behaviour	Ensure relevant Incident fields respond to configured conditions.
4	Reduce manual actions	Provide appropriate dynamic behaviour instead of unnecessary manual field management.
5	Validate implementation	Test the configured behaviour in the actual ServiceNow Incident form.
6	Document the project	Maintain configuration, screenshots, testing evidence and demonstration material.

6. Technologies and Platform

Technology / Component	Role in the Project
ServiceNow	Primary platform used to configure and demonstrate the project.
Incident Management	Main application area and form used for the implementation.
Client Script	Used to provide client-side behaviour according to the configured project requirement.
UI Policy	Used to dynamically control relevant field properties on the Incident form.

7. System Working Principle

The project follows a condition-based form interaction model. A user opens the Incident form and interacts with the relevant fields. When a configured condition is met, the appropriate Client Script or UI Policy behaviour is applied. The user can then observe the resulting field behaviour directly in the Incident form.

Stage	Process	Outcome
1	User opens the Incident form	Initial form is displayed.
2	User enters or changes relevant information	The configured condition can be evaluated.
3	Condition is applicable	Configured Client Script and/or UI Policy behaviour is triggered.
4	Field properties are controlled	Relevant fields display the configured behaviour.
5	User continues the Incident process	The form provides the updated interaction.

8. Client Script Documentation

Client Script is one of the main development components of the project. It is intended to provide client-side behaviour on the Incident form. The exact event, condition and script logic must follow the project requirement and the actual ServiceNow configuration.

- Identify the Incident table and relevant form fields.
- Create the Client Script with the required configuration.
- Define the appropriate client-side trigger or event specified for the implementation.
- Implement the required client-side behaviour.
- Save the configuration and verify it directly on the Incident form.
- Capture screenshots of the configuration and resulting behaviour for project evidence.

9. UI Policy Documentation

UI Policy is used in the project to dynamically control field properties in the Incident form. The policy is applied when its configured condition is satisfied and the associated UI Policy Actions determine the relevant field behaviour.

- Create the UI Policy for the Incident table.
- Define the condition required for the project.
- Configure the relevant UI Policy Actions.
- Save the policy and verify the behaviour from the Incident form.
- Test the form with the condition both applicable and not applicable where relevant.
- Capture screenshots as evidence.

10. Implementation Procedure

Step	Activity	Documentation Evidence
01	Open ServiceNow and navigate to Incident Management.	Incident module screenshot
02	Open the required Incident form.	Initial form screenshot
03	Identify relevant fields and conditions.	Requirement/configuration notes
04	Configure the Client Script.	Client Script configuration screenshot
05	Configure the UI Policy and required actions.	UI Policy configuration screenshot
06	Save the configurations.	Saved configuration evidence
07	Change the relevant field value or condition.	Execution screenshot
08	Observe the resulting field behaviour.	Final behaviour screenshot
09	Execute the test cases.	Testing evidence
10	Prepare final documentation and demonstration.	Final project files

11. Testing and Validation

Testing is performed to verify whether the configured Client Script and UI Policy behave as expected. The test process should begin with the initial Incident form and then evaluate the form after the relevant condition is changed. Each test case should record the expected result, actual result, Pass/Fail status and supporting evidence.

Test Area	What Should Be Verified
Incident Form	The form opens and displays the required fields.
Client Script	The configured client-side behaviour occurs under the applicable condition.
UI Policy	The configured field properties are applied when the policy condition is satisfied.
Condition Handling	Behaviour changes appropriately when the relevant condition changes.
Final Output	The resulting Incident form matches the intended project behaviour.

12. Project Benefits

Benefit	Description
Dynamic Form Behaviour	The Incident form can respond to configured conditions through Client Script and UI Policy.
Reduced Manual Actions	Relevant field behaviour can be controlled automatically according to configuration.
User-Friendly Interaction	The form can present appropriate field behaviour based on the current condition.
Structured Configuration	Client Script and UI Policy provide clearly defined configuration areas for the required behaviour.
Better Documentation	The implementation can be supported with screenshots, test cases and a complete project report.

13. Project Deliverables

Phase	Deliverable
01	Brainstorming and Ideation document
02	Requirement Analysis document
03	Project Design document, workflow and UI design
04	Project Planning document and task tracking
05	Project Development documentation, Client Script, UI Policy and screenshots

06	Test cases, test execution results and testing screenshots
07	Complete Project Documentation
08	Demo script and public demonstration video link

14. GitHub Repository Structure

The project is organized phase-wise so that the development history and supporting documents can be reviewed easily from the public GitHub repository.

servicenow-client-script-ui-policy-incident/

■■■■ README.md

■■■■ 01-Brainstorming-and-Ideation/

■■■■ 02-Requirement-Analysis/

■■■■ 03-Project-Design/

■ ■■■■ Project-Design.pdf

■ ■■■■ Workflow.png

■ ■■■■ UI-Design.png

■■■■ 04-Project-Planning/

■■■■ 05-Project-Development/

■ ■■■■ Client-Script/

■ ■■■■ UI-Policy/

■ ■■■■ Screenshots/

■■■■ 06-Project-Testing/

■■■■ 07-Project-Documentation/

■■■■ 08-Project-Demonstration/

15. Team Members

Role	Name
Team Leader	M Manisha
Member	M Monisha
Member	E Pattu Devi
Member	S Selva Abi
Member	M Sree Varshini

16. Project Outcome

The intended outcome of the project is a ServiceNow Incident form with dynamic, condition-based field behaviour implemented through Client Script and UI Policy. The completed implementation should be supported by configuration screenshots, testing evidence, documentation and a project demonstration.

The project also demonstrates the practical use of ServiceNow Administrator capabilities for configuring form behaviour. By organizing the work into brainstorming, requirements, design, planning, development, testing, documentation and demonstration phases, the complete project can be maintained and reviewed systematically.

17. Conclusion

Thus, the Client Script and UI Policy project provides a structured approach to managing dynamic behaviour in the ServiceNow Incident form. Client Script is used for client-side behaviour and UI Policy is used to dynamically control field properties according to configured conditions.

The project demonstrates how appropriate ServiceNow configuration can reduce unnecessary manual interaction and support a more user-friendly Incident form. The final implementation should be validated in the ServiceNow instance and supported by testing results before the project is submitted through the phase-wise GitHub repository.